

《现代C++编程实现》电子版: <https://leanpub.com/c01>

第10章 类模板

class template

YouTube频道: [hwdong](https://www.youtube.com/hwdong)

博客: hwdong-net.github.io

B站: [hw-dong](https://www.bilibili.com/channel/owner?id=hw-dong)

类模板

- 11.1 类模板
- 11.2 模板特化与偏特化

什么是类模板？

- 类模板是一个用于定义类的模式（模板），它本身不是类，而是在提供具体的模板参数后，编译器会基于类模板实例化出具体的

```
template<typename T1, typename T2>
class Pair {
public:
    T1 first;
    T2 second;

    Pair(const T1& a, const T2& b) : first(a), second(b) {}

    void display() const {
        std::cout << "First: " << first << ", Second: " << second << std::endl;
    }
};
```

类模板的实例化

- 类模板的实例化指从类模板生成具体的模板类；而类的实例化则指从模板类创建对象。

```
Pair<int, double> p(10, 3.14);
```

Pair<int, double> 指示编译器从类模板 Pair<T1, T2> 生成模板类 Pair<int, double>，此过程为**类模板的实例化**

p(10, 3.14) 通过构造函数创建 Pair<int, double> 的对象 p，此过程为**类的实例化**。

类模板的显式与隐式实例化

显式实例化：

- 在创建模板类对象时，可以显式指定模板参数，如
Pair<int,double> p(10, 3.14)，先生成特定模板类，再创建该类的对象。
- 另一种形式是显式模板实例化声明，如：

```
template class Pair<int, double>;
```

用于强制生成模板类代码，优化多文件编译。

类模板的显式与隐式实例化

隐式实例化：

- C++17 起支持类模板参数推导（Class Template Argument Deduction, CTAD），编译器可以根据构造函数参数自动推导模板参数，自动生成模板类并创建对象。例如：

```
Pair p(10, 3.14); // C++17+, 推导为 Pair<int, double>
```

等价于：

```
Pair<int, double> p(10, 3.14); // 显式指定
```

```
template<typename T1, typename T2>
class Pair {
public:
    T1 first;
    T2 second;
    Pair(const T1& a, const T2& b) : first(a), second(b) {}
    void display() const {
        std::cout << "First: " << first << ", Second: " << second << std::endl;
    }
};

int main() {
    Pair<int, double> p(10, 3.14); // 显式实例化
    p.display();

    Pair q("hwdong", "Author"); // 隐式实例化 (C++17+, 推导为 Pair<std::string, std::string>)
    q.display();

    template class Pair<char, double>; // 显式模板实例化, 优化编译
    return 0;
}
```

类模板的默认参数

- 类模板的模板参数可以指定默认值，从而在实例化模板时简化类型的书写

```
template<typename T = int>
class X {
public:
    T data;

    X(const T& d) : data(d) {}

    void show() const {
        std::cout << typeid(data).name() << '\t' << data << std::endl;
    }
};
```



```
template<typename T = int>
class X {
public:
    T data;

    X(const T& d) : data(d) {}

    void show() const {
        std::cout << typeid(data).name() << '\\t' << data << std::endl;
    }
};
```

```
int main() {
    X<> x(4.2); // 此处X<>不显式指定模板参数，默认使用int
    x.show();
    X y(4.2); // 从对象初始化参数4.2 隐式推断模板参数T是double
    y.show();
    return 0;
}
```

非类型模板参数

- 类模板还可以使用非类型模板参数。非类型模板参数是指在编译期确定的常量，它可以是整数、指针、引用等。

```
template <typename T, int Factor> // Factor 是非类型模板参数
class ScaledValue {
private:
    T value;

public:
    ScaledValue(T val) : value(val) {}

    T getScaled() const {
        return value * Factor;
    }
};
```

```
int main() {  
    // 创建一个用于将米转换为厘米的 ScaledValue  
    ScaledValue<double, 100> metersToCentimeters(2.5); // 2.5 米  
  
    // 创建一个用于将千克转换为克的 ScaledValue  
    ScaledValue<int, 1000> kilogramsToGrams(3); // 3 千克  
  
    std::cout << "2.5 meters in centimeters: "  
                << metersToCentimeters.getScaled() << std::endl; // 输出 250  
  
    std::cout << "3 kilograms in grams: "  
                << kilogramsToGrams.getScaled() << std::endl; // 输出 3000  
  
    return 0;  
}
```

定义类模板的成员函数

- 类模板 的成员函数可以在类模板内部定义，也可以在类模板外部定义。

在类模板外部定义成员函数时，需要注意以下几点：

- 必须在成员函数定义之前添加模板头，例如：`template <typename T>`。
- 定义时必须包含 **类的作用域**，即使用 `X<T>::` 来指定该成员函数属于类模板 `X`。
- `X<T>` 是类模板 `X` 的实例化形式，其中 `T` 是模板参数。

```
template <typename T>
class X {
    void f();
};

template <typename T>
void X<T>::f() {
    // 函数实现
}

int main() {
    return 0;
}
```

注意：常见的错误是将成员函数定义写成 `X::f()`，但是由于 `X` 是一个类模板，必须明确指定模板参数 `T`，因此正确的写法是 `X<T>::f()`。

11.2 模板特化与偏特化

模板特化为特定模板参数提供不同实现，适用于类型模板参数和非类型模板参数。**全特化**针对所有参数，**偏特化**针对部分参数。

全特化

- 全特化为模板的所有参数指定特定类型，提供与通用模板不同的实现。

```
// 全特化: Pair<bool, bool>
template<>
class Pair<bool, bool> {
public:
    bool first;
    bool second;
    Pair(bool a, bool b) : first(a), second(b) {}
    void display() const {
        cout << "First: " << (first ? "true" : "false")
              << ", Second: " << (second ? "true" : "false") << endl;
    }
};
```

// 通用模板

```
template<typename T1, typename T2>
```

```
class Pair {
```

```
public:
```

```
    T1 first;
```

```
    T2 second;
```

```
    Pair(const T1& a, const T2& b) : first(a), second(b) {}
```

```
    void display() const {
```

```
        cout << "First: " << first << ", Second: " << second << endl;
```

```
    }
```

```
};
```

// 全特化: Pair<bool, bool>

```
template<>
```

```
class Pair<bool, bool> {
```

```
public:
```

```
    bool first;
```

```
    bool second;
```

```
    Pair(bool a, bool b) : first(a), second(b) {}
```

```
    void display() const {
```

```
        cout << "First: " << (first ? "true" : "false")
```

```
        << ", Second: " << (second ? "true" : "false") << endl;
```

```
    }
```

```
};
```



```
int main() {  
    Pair<int, double> p1(1, 2.3);  
    p1.display();  
    Pair<bool, bool> p2(true, false);  
    p2.display();  
    return 0;  
}
```

偏特化

- 偏特化仅特化部分模板参数，仍为类模板。以下特化Pair的第一个参数为int*：

```
// 偏特化：只特化第一个模板参数为指针类型
template<typename T2>
class pair<int*, T2> { // 仅当第一个参数为指针类型时，进行偏特化
public:
    int* first;
    T2 second;
    pair(int* f, T2 s) : first(f), second(s) {}
};
```

非类型模板参数特化

- 非类型模板参数也可特化。以下为ScaledValue的全特化，当Factor=0时返回0：

```
template<typename T, int Factor>
class ScaledValue {
public:
    T value;
    ScaledValue(T val) : value(val) {}
    T getScaled() const { return value * Factor; }
};
```

```
template<typename T>
class ScaledValue<T, 0> {
public:
    T value;
    ScaledValue(T val) : value(val) {}
    T getScaled() const { return T(0); }
};
```

11.3. 类模板的友元

类模板可使用 `friend` 关键字声明普通函数、类、函数模板或类模板为友元，授予访问私有成员的权限。

普通友元函数 或 友元类

- 类模板可使用 friend 关键字声明普通函数、类、函数模板或类模板为友元，授予访问私有成员的权限。

```
template<typename T>
class X {
    int a = 0; // C++11+ initialization
    friend void fun();
    friend class A;
};
```

```
template<typename T>
class X {
    int a = 0; // C++11+ initialization
    friend void fun();
    friend class A;
};
```

```
class A {
public:
    void access(X<int>& x) { cout << "A accesses: " << x.a << endl; }
};
```

```
void fun() {
    X<int> x;
    X<double> y;
    cout << x.a << '\t' << y.a << endl;
}
```

```
int main() {
    fun(); // output: 0    0
    A a;
    X<int> x;
    a.access(x); // output: A accesses: 0
    return 0;
}
```

友元函数模板

- 类模板的**友元函数模板**可使用类模板的模板参数或独立模板参数。

示例1：使用相同模板参数

```
template<typename T>
class Y {
    // 声明友元函数模板
    friend Y<T>* g(Y<T>& e);
};
```

```
template<typename T>
Y<T>* g(Y<T>& e) {
    Y<T>* p = new Y<T>(e);
    return p;
}
```

示例1：使用相同模板参数

```
template<typename T>
class Y {
    // 声明友元函数模板
    friend Y<T>* g(Y<T>& e);
};
```

```
template<typename T>
Y<T>* g(Y<T>& e) {
    Y<T>* p = new Y<T>(e);
    return p;
}
```

```
int main() {
    Y<int> e(10);
    auto p = g<int>(e); // 正确: g<int> 是 Y<int> 的友元
    std::cout << p->data << std::endl;

    Y<double> x(20);
    auto q = g<int>(x); // 错误: g<int> 不是 Y<double> 的友元
    std::cout << q->data << std::endl; // 编译错误
    return 0;
}
```


示例2：独立模板参数

```
// 定义类模板 A，模板参数为 T
template<typename T>
class A {
    T value;

    // 将不相关的模板参数类型 U 作为参数的函数声明为友元
    template<typename U>
    friend void show(U& obj);
public:
    A(T v) : value(v) {}
};
```

```
// 友元函数模板
template<typename U>
void show(U& obj) {
    // 由于 show 是 A<T> 的友元，即使 U 不是 A<T>，它仍然可以访问 A<T> 的私有成员
    std::cout << obj.value << std::endl;
}
```

```
int main() {
    A<int> a(42);
    show(a); // 访问 A<int> 的私有成员 value
    return 0;
}
```

返回类型涉及类模板的实例

```
template<typename T>
class A {
    T value;

    // 友元函数的模板参数 U 与 A<T> 的 T 不同
    template<typename U>
    friend A<U> create(U v);

public:
    A(T v) : value(v) {}

    void show() const {
        std::cout << "value: " << value << std::endl;
    }
};
```

```
// 友元函数模板，创建并返回 A<U> 类型对象
template<typename U>
A<U> create(U v) {
    A<U> obj(v); // 访问 A<U> 的私有成员
    return obj;
}
```

```
int main() {
    A<int> a = create(42); // 这里 A<int> 作为返回值
    a.show(); // 正确访问
    return 0;
}
```

类模板作为友元

- 类模板也可以作为其他类模板的友元。类模板Y<T>或Z<U>可作为X<T>的友元，授予访问私有成员的权限。

```
template<typename T>
class X {
public:
    X(const T& value) : data(value) {}
    friend class Y<T>; // Y<T>为友元
private:
    T data;
};
```

```
// 提前声明Y，因为X<T>的友元声明中需要Y<T>
template<typename T>
class Y;

template<typename T>
class X {
public:
    X(const T& value) : data(value) {}
    friend class Y<T>; // Y<T>为X<T>的友元
    template<typename U>
    friend class Z; // Z<U>为所有X<T>的友元
private:
    T data;
};
```

```
template<typename T>
class Y {
public:
    void show(const X<T>& w) {
        cout << "X包含:  " << w.data << endl;
    }
};
```

```
template<typename U>
class Z {
public:
    void access(const X<U>& x) {
        cout << "Z访问:  " << x.data << endl;
    }
};
```

```
int main() {  
    X<int> x(42);  
    X<double> z(4.2);  
    Y<int> y;  
    y.show(x); // 输出: X包含: 42  
    // y.show(z); // 错误: Y<int>无权访问X<double>的私有成员data  
  
    Z<double> z_obj;  
    z_obj.access(z); // 输出: Z访问: 4.2  
    return 0;  
}
```

11.4 类模板与继承

- 类模板与继承结合：

类模板的继承

- 可以让一个类模板从另一个类模板继承，从而复用基类模板的功能，同时扩展新的行为。

```
template<typename T>
class SpecialBox : public Box<T> { // 继承自 Box<T>
public:
    SpecialBox(const T& c) : Box<T>(c) {} // 调用基类构造函数

    void showSpecialContent() const {
        std::cout << "Special Box contains: " << this->content << std::endl;
    }
};
```

```
template<typename T>
class Box {
public:
    T content;

    Box(const T& c) : content(c) {}

    void showContent() const {
        std::cout << "Box content: " << content << std::endl;
    }
};
```

```
template<typename T>
class SpecialBox : public Box<T> { // 继承自 Box<T>
public:
    SpecialBox(const T& c) : Box<T>(c) {} // 调用基类构造函数

    void showSpecialContent() const {
        std::cout << "Special Box contains: " << this->content << std::endl;
    }
};
```



```
template<typename T>
class SpecialBox : public Box<T> { // 继承自 Box<T>
public:
    SpecialBox(const T& c) : Box<T>(c) {} // 调用基类构造函数

    void showSpecialContent() const {
        std::cout << "Special Box contains: " << this->content << std::endl;
    }
};
```

```
int main() {
    SpecialBox<int> sb(42);
    sb.showContent(); // 继承自 Box
    sb.showSpecialContent(); // 特有方法
}
```

关于 this-> 的必要性

- 在 SpecialBox<T> 内部，基类 Box<T> 的成员 content 不能直接使用，而需要 this->content 访问。这是因为 Box<T> 也是一个模板，编译器在解析 SpecialBox<T> 时尚未解析 Box<T> 的具体实例，所以不会直接查找 content，它被认为是非依赖名（**non-dependent name**），不会在基类模板中查找。

```
void showSpecialContent() const {  
    std::cout << "Special Box contains: " << this->content << std::endl;  
}
```

关于 this-> 的必要性

- 如果不想使用 this->, 可以用 using 关键字让基类成员在派生类中可见:

```
template<typename T>
class SpecialBox : public Box<T> {
public:
    using Box<T>::content;    // 让基类成员可见
    using Box<T>::showContent; // 让基类成员可见
};
```

继承不同模板参数的基类

- 可以让派生类模板的参数与基类模板的参数不同。

```
template<typename T, typename U>
class SpecialBox : public Box<U> { // 继承 Box<U>
    T extraValue;
public:
    SpecialBox(const U& c, const T& extra) : Box<U>(c), extraValue(extra) {}

    void showSpecialContent() const {
        std::cout << "Box content: " << this->content
                    << ", Extra value: " << extraValue << std::endl;
    }
};
```

```
template<typename T, typename U>
class SpecialBox : public Box<U> { // 继承 Box<U>
    T extraValue;
public:
    SpecialBox(const U& c, const T& extra) : Box<U>(c), extraValue(extra) {}

    void showSpecialContent() const {
        std::cout << "Box content: " << this->content
                    << ", Extra value: " << extraValue << std::endl;
    }
};
```

```
int main() {
    SpecialBox<std::string, int> sb(42, "Extra");
    sb.showContent(); // 继承自 Box<int>
    sb.showSpecialContent(); // 访问 SpecialBox 的成员
}
```

继承模板成员函数

- 在模板类中，如果成员函数本身是模板函数，那么派生类也可以继承这些成员模板函数，并且模板函数可以在派生类中被使用。

```
template<typename T>
class Box {
public:
    T value;
    Box(T v) : value(v) {}

    template<typename U>
    void display(U extra) const {
        std::cout << "Box value: " << value << ", Extra: " << extra << std::endl;
    }
};
```

```
template<typename T>
class Box {
public:
    T value;
    Box(T v) : value(v) {}

    template<typename U>
    void display(U extra) const {
        std::cout << "Box value: " << value << ", Extra: " << extra << std::endl;
    }
};
```

```
template<typename T>
class SpecialBox : public Box<T> {
public:
    SpecialBox(T v) : Box<T>(v) {}

    void show() const {
        this->display("Hello, world!"); // 调用继承的模板成员函数
    }
};
```

```
template<typename T>
class Box {
public:
    T value;
    Box(T v) : value(v) {}

    template<typename U>
    void display(U extra) const {
        std::cout << "Box value: " << value << ", Extra: " << extra << std::endl;
    }
};
```

```
template<typename T>
class SpecialBox : public Box<T> {
public:
    SpecialBox(T v) : Box<T>(v) {}

    void show() const {
        this->display("Hello, world!"); // 调用继承的模板成员函数
    }
};

int main() {
    SpecialBox<int> sb(10);
    sb.show(); // Box value: 10, Extra: Hello, world!
}
```


Vector 类模板

`Vector` 类模板使用模板参数 `T` 表示元素类型，提供以下功能：

- 添加元素到末尾（`push_back`）。
- 通过下标访问元素（`operator[]`）。
- 查询元素数量（`getSize`）和内存容量（`getCapacity`）。
- 打印所有元素（`print`）。

```
template <typename T>
class Vector {
private:
    T* data;          // 动态数组指针
    size_t size;      // 元素数量
    size_t capacity;  // 内存容量

public:
    // 构造函数
    Vector() : data(nullptr), size(0), capacity(0) {}

    // 拷贝构造函数
    Vector(const Vector& other)
        : data(new T[other.capacity]), size(other.size), capacity(other.capacity) {
        for (size_t i = 0; i < size; ++i) {
            data[i] = other.data[i];
        }
    }
}
```

// 拷贝赋值运算符

```
Vector& operator=(const Vector& other) {  
    if (this != &other) {  
        delete[] data;  
        data = new T[other.capacity];  
        size = other.size;  
        capacity = other.capacity;  
        for (size_t i = 0; i < size; ++i) {  
            data[i] = other.data[i];  
        }  
    }  
    return *this;  
}
```

// 析构函数

```
~Vector() {  
    delete[] data;  
}
```

// 获取元素数量

```
size_t getSize() const {  
    return size;  
}
```

// 获取容量

```
size_t getCapacity() const {  
    return capacity;  
}
```

// 添加元素

```
void push_back(const T& value) {  
    if (size == capacity) {  
        capacity_ = (capacity == 0) ? 1 : capacity * 2;  
        T* temp = new T[capacity_];  
        try {  
            for (size_t i = 0; i < size; ++i) {  
                temp[i] = data[i];  
            }  
        } catch (...) {  
            delete[] temp;  
            throw;  
        }  
        delete[] data;  
        data = temp;  
    }  
    data[size] = value;  
    ++size;  
    capacity = capacity_;  
}
```

// 访问元素

```
T& operator[](size_t index) {  
    if (index >= size) {  
        throw std::out_of_range("索引越界");  
    }  
    return data[index];  
}
```

```
const T& operator[](size_t index) const {  
    if (index >= size) {  
        throw std::out_of_range("索引越界");  
    }  
    return data[index];  
}
```

// 打印元素

```
void print() const {  
    for (size_t i = 0; i < size; ++i) {  
        std::cout << data[i] << " ";  
    }  
    std::cout << std::endl;  
}  
};
```

```
int main() {  
    Vector<int> vec;  
    vec.push_back(1);  
    vec.push_back(2);  
    vec.push_back(3);  
  
    std::cout << "元素数量: " << vec.getSize() << std::endl; // 输出: 3  
    std::cout << "容量: " << vec.getCapacity() << std::endl; // 输出: 4  
    vec.print(); // 输出: 1 2 3  
  
    std::cout << "索引1的元素: " << vec[1] << std::endl; // 输出: 2  
  
    try {  
        std::cout << vec[10] << std::endl; // 触发异常  
    } catch (const std::out_of_range& e) {  
        std::cout << "错误: " << e.what() << std::endl; // 输出: 错误: 索引越界  
    }  
  
    Vector<int> vec2 = vec; // 测试拷贝  
    vec2.print(); // 输出: 1 2 3  
  
    return 0;  
}
```


关注

Youtube频道: <https://www.youtube.com/c/hwdong>

博客: <https://hwdong-net.github.io>